Compte-Rendu

Architecture 3-Tiers

Mathis PAOLINI — 2SIO

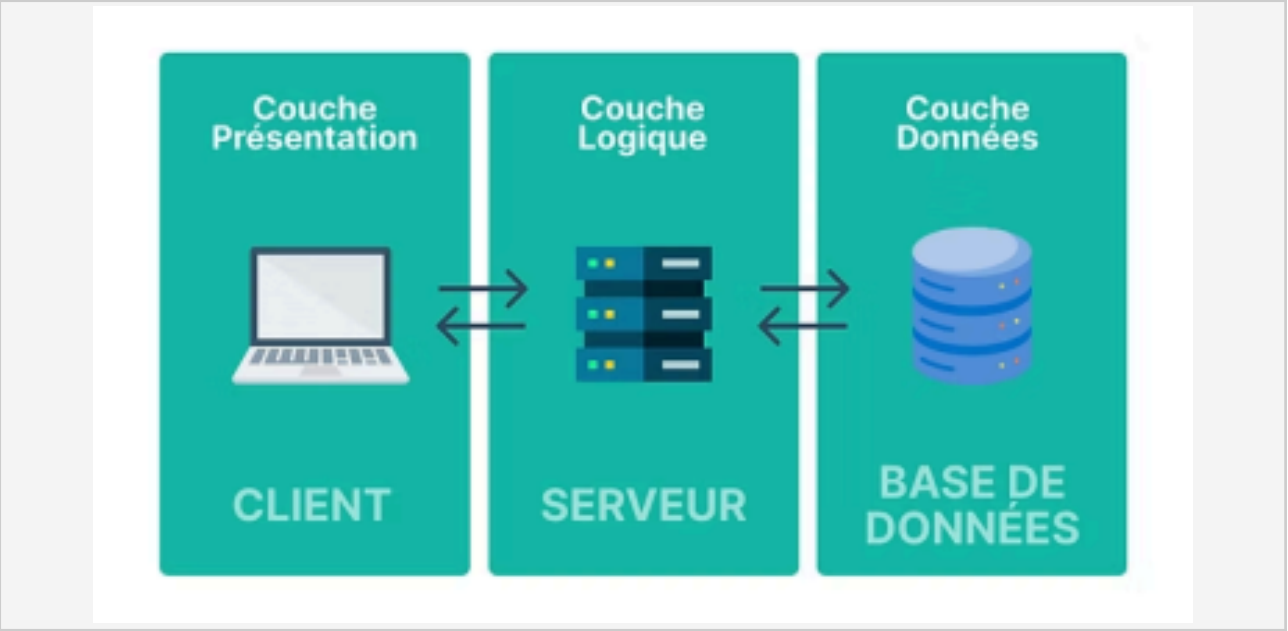
OS	Version OS	Date	Version Procédure
Debian	12.5	08/10/2025	V1

Pré-requis

- Deux VMs Debian 12 ou supérieure
- VM Web (172.17.21.X) : installation d'Apache2 et PHP8
- VM BDD (172.17.21.Y) : installation de MariaDB
- Un schéma d'architecture avec IP fixe

Sommaire

- 1. Introduction
- 2. Configuration MariaDB pour l'accès distant
- 3. Création d'un utilisateur MariaDB
- 4. Configuration du serveur Web
- 5. Installation et configuration de phpMyAdmin
- 6. Validation et tests



1. Introduction

Une architecture 3-tiers (ou architecture à trois niveaux) est un modèle d'architecture logicielle client-serveur qui sépare les fonctions de l'application en trois couches logiques et physiques distinctes, facilitant ainsi le développement, la maintenance et la scalabilité.

Chaque niveau est spécialisé dans une fonction précise et communique uniquement avec les niveaux adjacents :

1 — Le niveau présentation (tier présentation)

- **Rôle :** l'Interface Homme Machine (IHM) qui gère l'affichage des informations et la capture des saisies de l'utilisateur.
- **Technologies :** navigateur Web (HTML, CSS, JavaScript) ou application cliente lourde.
- **Localisation physique :** le poste client (ordinateur, smartphone, tablette).

2 — Le niveau application (tier application / logique métier)

- **Rôle :** il contient la logique métier de l'application (règles de gestion, calculs, workflows). Il sert d'intermédiaire entre le niveau présentation et le niveau données.
- **Technologies :** serveur Web (Apache, Nginx) et langages de programmation côté serveur (PHP, Java/JEE, Python, Node.js).
- **Localisation physique :** un ou plusieurs serveurs d'application.

3 — Le niveau données (tier données / persistance)

- **Rôle :** stockage, gestion et accès aux données. Il est uniquement interrogé par le niveau application.
- **Technologies :** Système de Gestion de Base de Données (SGBD) comme MariaDB, MySQL, PostgreSQL, Oracle, etc.
- **Localisation physique :** un ou plusieurs serveurs de base de données.

2. Configuration MariaDB pour l'accès distant

Sur la VM BDD, installer mariadb-server :

```
apt install mariadb-server -y
```

Sur la VM MariaDB, modifier le fichier de configuration :

```
apt install mariadb-server
```

```
nano  /etc/mysql/mariadb.conf.d/50-server.cnf
```

Remplacer bind-address par 0.0.0.0, puis redémarrer le service :

```
systemctl restart mariadb
```

3. Création d'un utilisateur MariaDB

Lancer MariaDB en tapant :

```
mariadb
```


```
mariadb
```


Création de l'utilisateur avec son mot de passe :

```
GRANT ALL PRIVILEGES ON   t_films.* TO 'titi'@'%' IDENTIFIED BY 'password123';
```

GRANT ALL PRIVILEGES ON → donne des droits sur la base.

Rafraîchir les droits sur la base de données :

```
FLUSH PRIVILEGES;
```

4. Configuration du serveur Web

Sur la VM Web, installer Apache2 et PHP avec le module MySQL :

```
apt install apache2 php php-mysql -y
```

```
apt install apache2 php libapache2-mod-php php-mysql -y
```

Tester la connexion distante avec :

```
mysql -u titi -h 172.17.21.Y -p
```

5. Installation et configuration de phpMyAdmin

Sur la VM Web, installer phpMyAdmin :

```
apt install phpmyadmin -y
```

apt install phpmyadmin -y

Choisir Apache2 comme serveur web lors de l'installation.

Modifier /etc/phpmyadmin/config.inc.php :

```
nano /etc/phpmyadmin/config.inc.php
```

Remplacer localhost par l'adresse IP de la VM BDD (172.17.21.Y).

systemctl restart apache2

Redémarrer Apache :

```
systemctl restart apache2
```

6. Validation et tests

6.1 — Accès à phpMyAdmin

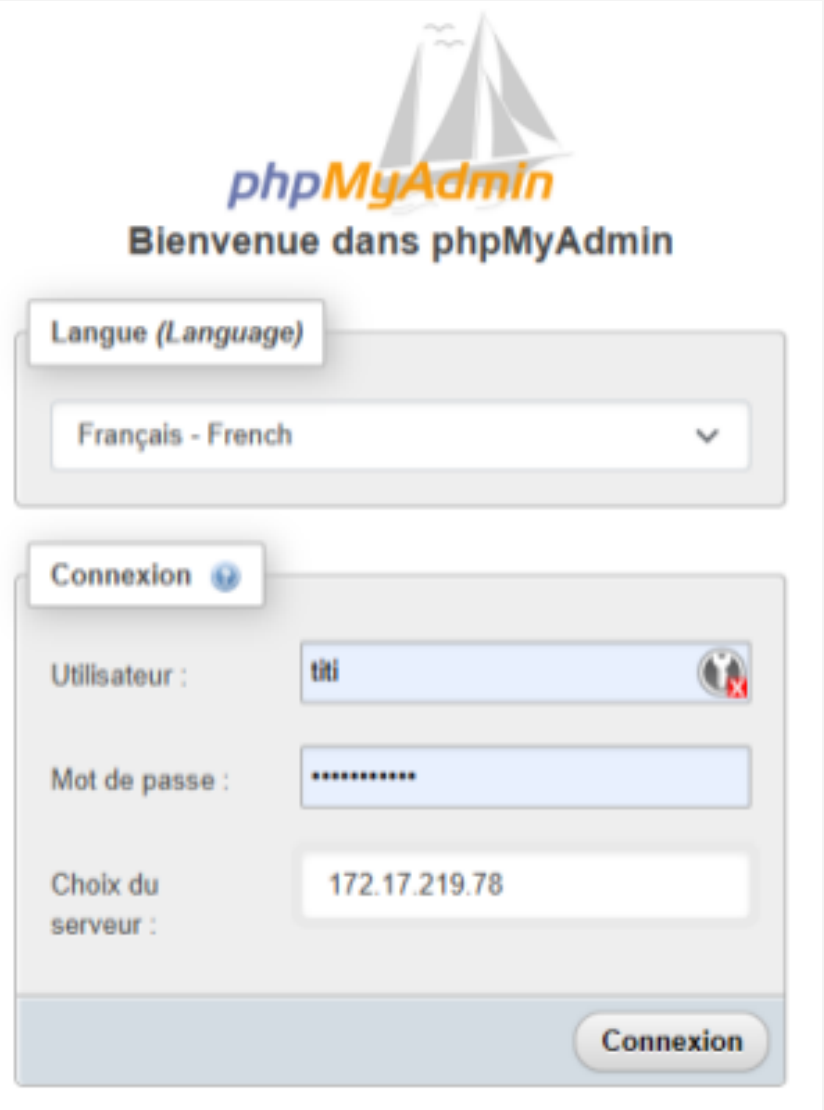
Accéder à phpMyAdmin via l'URL suivante depuis le client :

```
http://172.17.21.X/phpmyadmin
```

Se connecter avec les identifiants mis en place en amont.

⚠ **ATTENTION : il faut bien choisir le bon serveur lors de la connexion.**

```
Utilisateur : titi
Mot de passe : password123
```



6.2 — Test de connexion PHP

Créer un fichier db_test.php dans /var/www/html/ :

```php
<?php
// Configuration de la base de données
$servername = "172.17.21.Y";
$database = "t_films";
$username = "titi";
$password = "password123";
// Connexion à la base de données
$conn = mysqli_connect($servername, $username, $password, $database);
if (!$conn) {
die("Echec de connexion" . mysqli_connect_error());
}
echo "Connexion réussie";
mysqli_close($conn);
?>
```

Accéder via :

```
http://172.17.21.X/db_test.php
```

Si le message "Connexion réussie" apparaît, l'architecture fonctionne correctement.

Conclusion

Ce TP permet de comprendre et de mettre en place une architecture logicielle en 3-tiers. Il illustre l'importance de la séparation des couches (présentation, application et données) pour améliorer la sécurité, la modularité et la scalabilité.

Ce TP a permis de concevoir une architecture **3-tiers** en isolant les couches de présentation, d'application (**Apache/PHP**) et de données (**MariaDB**) sur des serveurs distincts. Nous avons configuré l'accès distant à la base de données, sécurisé les privilèges utilisateurs et déployé **phpMyAdmin** pour l'administration. Cette structure modulaire garantit une meilleure sécurité, une maintenance simplifiée et une plus grande flexibilité pour l'évolution des services web.